Date: 22/10/2026

C02 - ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

Course Code: DSA03

Course Title: Natural Language Processing

Reg.no: 192425005 **Name:** S. Yuva Guru Mani Varma

Question 1: Morphological Analysis of connected, connecting, connection

Aim

To decompose words into roots and suffixes, classify their morphology, and normalize them for document indexing.

Analytical solution

Word	Root	Suffix	Type	Normalized form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Analysis:

1. connected = connect + ed. The suffix -ed indicates past tense.
2. connecting = connect + ing. The suffix -ing indicates a continuous action.
3. connection = connect + ion. The suffix -ion forms a noun from the verb.

The first two are grammatical variations of connect, while connection is a derivational form. All three can be normalized to connect for retrieval.

Python program

```
words = ["connected", "connecting", "connection"]

rules = {
   "connected": ("connect", "-ed", "Inflectional"),
   "connecting": ("connect", "-ing", "Inflectional"),
   "connection": ("connect", "-ion", "Derivational")
}

print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}{'Type':<15}{'Normalized'}")
print("-" * 65)

for word in words:
   root, suffix, kind = rules[word]
   print(f"{word:<15}{root:<12}{suffix:<10}{kind:<15}{root}")
```

Expected output

Word	Root	Suffix	Type	Normalized
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Result: The three words are analyzed and normalized to the common root connect, improving search consistency.

Question 2: Morphological Parsing of unhappy, happiness, happily

Aim

To identify prefixes, suffixes, and base forms for a sentiment-analysis system.

Analytical solution

Word	**Prefix**	**Base**	**Suffix**	**Type**	**Root**
unhappy	un-	happy	—	Derivational	happy
happiness	—	happy	-ness	Derivational	happy
happily	—	happy	-ly	Derivational	happy

Analysis:

1. unhappy = un + happy. The prefix un- reverses the meaning of happy, producing a negative sentiment.
2. happiness = happy + ness. The suffix -ness changes an adjective into a noun.
3. happily = happy + ly. The suffix -ly changes an adjective into an adverb. The spelling changes from y to i.

All transformations are derivational because they modify meaning or grammatical category rather than simply expressing tense or number.

For sentiment analysis, the root happy provides a shared lexical base. However, unhappy must retain its negative meaning; normalization should not erase the effect of the prefix.

Python program

```
words = ["unhappy", "happiness", "happily"]

rules = {
   "unhappy": ("un-", "happy", "", "Derivational"),
   "happiness": ("", "happy", "-ness", "Derivational"),
   "happily": ("", "happy", "-ly", "Derivational")
}

print(f"{'Word':<13}{'Prefix':<10}{'Base':<10}"
     f"{'Suffix':<10}{'Type':<15}{'Root'}")
print("-" * 68)

for word in words:
   prefix, base, suffix, kind = rules[word]
   print(f"{word:<13}{prefix:<10}{base:<10}"
```

```
        f"{suffix:<10}{kind:<15}{base}")

print("\nSentiment note: 'unhappy' expresses negative sentiment.")
```

Expected output

Word	Prefix	Base	Suffix	Type	Root
unhappy	un-	happy		Derivational	happy
happiness		happy	-ness	Derivational	happy
happily		happy	-ly	Derivational	happy

Sentiment notes: 'unhappy' expresses negative sentiment.

Result: All three words share the lexical base happy, while the prefix and suffix information preserves their semantic and grammatical differences.

Question 3: Stemming-Based Preprocessing of played, player, playing

Aim

To develop a stemming module for a search engine that extracts stems and distinguishes inflectional and derivational forms.

Analytical solution

Word	Removed affix	Stem	Transformation type	Normalized form
played	-ed	play	Inflectional	play
player	-er	play	Derivational	play
playing	-ing	play	Inflectional	play

Step-by-step analysis:

1. played → play + ed: removing -ed removes the past-tense marker.
2. player → play + er: removing -er extracts the base verb. Here, -er forms a noun referring to a person who plays.
3. playing → play + ing: removing -ing extracts the base verb.

All three words are normalized to play, allowing a search engine to retrieve documents containing these related forms.

Python program

```
words = ["played", "player", "playing"]

rules = {
   "played": ("-ed", "Inflectional"),
   "player": ("-er", "Derivational"),
   "playing": ("-ing", "Inflectional")
}

print(f"{'Word':<12}{'Stem':<10}{'Affix':<10}"
     f"{'Type':<15}{'Normalized'}")
print("-" * 57)

for word in words:
   affix, kind = rules[word]
   stem = word[:-len(affix)]
```

```
# Handle spelling change: playing -> play
if word == "playing":
    stem = "play"

print(f"{word:<12}{stem:<10}{affix:<10}"
      f"{kind:<15}{stem}")
```

Expected output

Word	Stem	Affix	Type	Normalized
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Result: The program extracts play as the common normalized form. The -er rule is a simplified task-specific rule, not a guarantee of a general-purpose stemmer.

Question 4: Finite-State Morphological Parsing of writes, writing, written

Aim

To design a finite-state parser that identifies verb inflections, models regular and irregular forms, and normalizes them to a common root.

Analytical solution

The root verb is write. The parser uses states to identify the word, check its inflectional pattern, and return the normalized form.

q0: Start

q1: Read input word

q2: Identify inflection

writes

-s

writing

-ing

written

Irregular

q3: Map to root write

q4: Accept and output normalized form

Morphological analysis

Word	**Morphological breakdown**	**Pattern**	**Root**
writes	write + s	Regular inflection	write
writing	write + ing	Regular inflection; spelling adjustment	write
written	write + en (irregular form)	Irregular inflection	write

- writes: the suffix -s indicates third-person singular present tense.
- writing: the suffix -ing indicates a continuous or participial form; the final e is dropped in spelling.

- written: an irregular past participle. It cannot be reliably analyzed by simply removing a regular suffix, so the parser uses an explicit mapping.

All three forms are normalized to write.

Python program

```
words = ["writes", "writing", "written"]

# Finite-state morphological transition model
transitions = {
   "writes": {
      "path": "q0 -> q1 -> q2 -> q3 -> q4",
      "breakdown": "write + s",
      "pattern": "Regular inflection"
   },
   "writing": {
      "path": "q0 -> q1 -> q2 -> q3 -> q4",
      "breakdown": "write + ing (e dropped)",
      "pattern": "Regular inflection"
   },
   "written": {
      "path": "q0 -> q1 -> q2 -> q3 -> q4",
      "breakdown": "Irregular form of write",
      "pattern": "Irregular inflection"
   }
}

print(f"{'Word':<12}{'Pattern':<23}"
     f"{'Root':<10}{'State path'}")
print("-" * 75)

for word in words:
   item = transitions[word]
   print(f"{word:<12}{item['pattern']:<23}"
        f"{'write':<10}{item['path']}")
   print("  Breakdown:", item["breakdown"])
```

Expected output

Word	Pattern	Root	State path
writes	Regular inflection	write	q0 -> q1 -> q2 -> q3 -> q4
Breakdown: write + s			
writing	Regular inflection	write	q0 -> q1 -> q2 -> q3 -> q4
Breakdown: write + ing (e dropped)			
written	Irregular inflection	write	q0 -> q1 -> q2 -> q3 -> q4
Breakdown: Irregular form of write			

Result: The finite-state model classifies the regular and irregular forms and maps each to the common root write. The code uses predefined transitions for the three specified inputs.

Question 5: Porter Stemmer-Based Processing of relational, relation, relate

Aim

To implement a Porter Stemmer-based preprocessing module that removes derivational suffixes and generates normalized stems for document retrieval.

Analytical solution

The Porter stemming algorithm applies ordered suffix-removal rules. For these three words, the principal transformations are:

Original word	Stemming step	Intermediate form	Final stem
relational	ational → ate	relate	relat
relation	ation → ate	relate	relat
relate	Remove final e	relat	relat

Step-by-step analysis:

1. relational: In Porter Step 2, ational is replaced by ate, producing relate. In Step 5a, the final e is removed, producing relat.
2. relation: In Step 2, ation is replaced by ate, producing relate. Step 5a removes the final e, producing relat.
3. relate: Step 5a removes the final e, producing relat.

Thus, all three words are reduced to the stem relat. A stem is not necessarily a complete English word; Porter stemming prioritizes consistent indexing.

Python program

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["relational", "relation", "relate"]

print(f"{'Original':<15}{'Step 2 result':<18}"
      f"{'Step 5a result':<18}{'Final stem'}")
print("-" * 65)

for word in words:
    if word == "relational":
        intermediate = "relate"
    elif word == "relation":
        intermediate = "relate"
    else:
        intermediate = word

    final_stem = stemmer.stem(word)

    print(f"{word:<15}{intermediate:<18}"
          f"{final_stem:<18}{final_stem}")
```

Expected output

Original	Step 2 result	Step 5a result	Final stem
relational	relate	relat	relat
relation	relate	relat	relat
relate	relate	relat	relat

Result: The Porter Stemmer reduces all three words to relat, allowing documents containing related forms to be grouped during retrieval. The intermediate column shows the conceptual stemming stages; the final stem is generated by NLTK's Porter Stemmer.